# Greedy regret heuristics

Adam Mierzwicki 160318, Marcin Perek 160298

## Table of contents

1. Problem description
2. Pseudocode
3. Results
4. Best solutions
5. Best solutions - node indices
6. Information whether the best solutions have been checked with the solution checker.
7. (Link to) the source code
8. Conclusions

## 1. Problem description

We are given three columns of integers with a row for each node. The first two columns contain x and y coordinates of the node positions in a plane. The third column contains node costs. The goal is to select exactly 50% of the nodes (if the number of nodes is odd we round the number of nodes to be selected up) and form a Hamiltonian cycle (closed path) through this set of nodes such that the sum of the total length of the path plus the total cost of the selected nodes is minimized. The distances between nodes are calculated as Euclidean distances rounded mathematically to integer values.

We implement four heuristics based on 2-regret:

1. **Nearest neighbor (any position) 2-regret**: start from one node. In each iteration, for every remaining node compute the best and second-best insertion position in the current path (including before the first, after the last node and in the middle). Insert the node with the largest regret (second best - best) at its best position.
2. **Greedy cycle 2-regret**: start from one node and its cheapest neighbor, forming a two-node cycle. Then repeat the 2-regret selection from method 1, but nodes can be inserted into any edge of the cycle, including the closing edge between the last and first node, instead of only into the path or at its ends.
3. **Nearest neighbor (any position) weighted 2-regret**: as in 1, but the node is chosen by maximizing `w_regret * regret - w_obj * best_insertion_cost`.
4. **Greedy cycle weighted 2-regret**: as in 2, but the node is chosen by maximizing `w_regret * regret - w_obj * best_insertion_cost`.

## 2. Pseudocode

Nearest neighbor (any position) 2-regret

```
select starting node
add starting node to solution
delete starting node from remaining nodes
repeat
    for node in remaining nodes
        initialize best cost, second best cost to infinity
        for position in insert positions in solution (before first, between
nodes, after last)
            if position is before first / after last node
                distance = distance from node to first/last node
            else
                distance = distance from node before position to node +
distance from node to node after position
                           - distance from node before position to node
after position
            total cost = distance + node cost
            record and update best cost, second best cost and best position
for this node
        regret = second best cost - best cost
    choose node with the highest regret (on ties choose the one with the
lowest best insertion cost)
    insert chosen node into its best position in solution
    delete chosen node from remaining nodes
until solution contains ceil(n/2) nodes
```

Greedy cycle 2-regret

```
select starting node
add starting node to solution
delete starting node from remaining nodes
find node with the lowest (distance from starting node + node cost) among
remaining nodes
add it to solution
delete it from remaining nodes
repeat
    for node in remaining nodes
        initialize best cost and second best cost to infinity
        for position in insert positions in solution (between nodes,
including closing edge between last and first node)
            distance = distance from node before position to node +
distance from node to node after position
                       - distance from node before position to node after
position
            total cost = distance + node cost
            record and update best cost, second best cost and best position
for this node
        regret = second best cost - best cost
    choose node with the highest regret (on ties choose the one with the
lowest best insertion cost)
    insert chosen node into its best position in solution
    delete chosen node from remaining nodes
until solution contains ceil(n/2) nodes
```

Nearest neighbor (any position) weighted 2-regret

```
select starting node
add starting node to solution
delete starting node from remaining nodes
repeat
    for node in remaining nodes
        initialize best cost and second best cost to infinity
        for position in insert positions in solution (before first, between
nodes, after last)
            if position is before first / after last node
                distance = distance from node to first/last node
            else
                distance = distance from node before position to node +
distance from node to node after position
                           - distance from node before position to node
after position
            total cost = distance + node cost
            record and update best cost, second best cost and best position
for this node
        regret = second best cost - best cost
        score = w_regret * regret - w_obj * best cost
    choose node with the highest score
    insert chosen node into its best position in solution
    delete chosen node from remaining nodes
until solution contains ceil(n/2) nodes
```

Greedy cycle weighted 2-regret

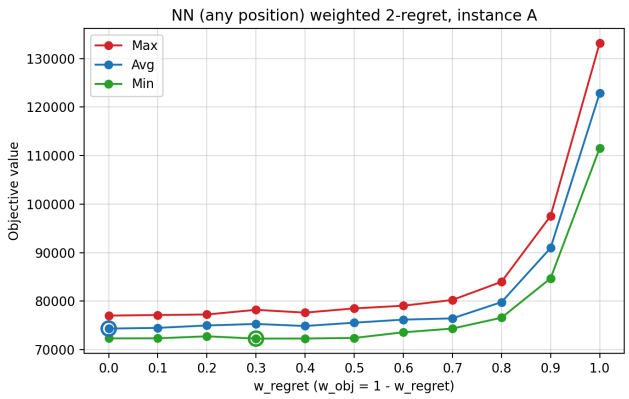
```
select starting node
add starting node to solution
delete starting node from remaining nodes
find node with the lowest (distance from starting node + node cost) among
remaining nodes
add it to solution
delete it from remaining nodes
repeat
    for node in remaining nodes
        initialize best cost and second best cost to infinity
        for position in insert positions in solution (between nodes,
including closing edge between last and first node)
            distance = distance from node before position to node +
distance from node to node after position
                       - distance from node before position to node after
position
            total cost = distance + node cost
            record and update best cost, second best cost and best position
for this node
        regret = second best cost - best cost
        score = w_regret * regret - w_obj * best cost
    choose node with the highest score
    insert chosen node into its best position in solution
    delete chosen node from remaining nodes
until solution contains ceil(n/2) nodes
```

# 3. Results

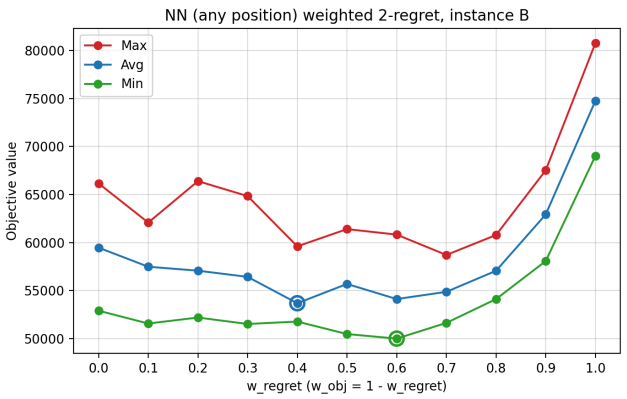
| Method | Instance | | | | | |
|---|---|---|---|---|---|---|
| | A | | | B | | |
| | Min | Avg | Max | Min | Avg | Max |
| **(lab1) Random** | 236589 | 261052.35 | 292354 | 182493 | 206729.01 | 234208 |
| **(lab1) Nearest neighbor** | 84362 | 87476.63 | 90075 | 56007 | 59397.335 | 63166 |
| **(lab1) Nearest neighbor (any position)** | 72328 | 74306.53 | 77022 | 52915 | 59476.225 | 66164 |
| **(lab1) Greedy cycle** | 72590 | 73758.44 | **75345** | 53016 | 59832.475 | 63736 |
| **Nearest neighbor (any position) 2-regret** | 110950 | 121360 | 135771 | 68761 | 73649.8 | 79756 |
| **Greedy cycle 2-regret** | 110303 | 120368 | 128089 | 67078 | 72368 | 76382 |
| **Nearest neighbor (any position) weighted 2-regret** | 72419 | 75561.2 | 78504 | **50485** | **55698.3** | **61406** |
| **Greedy cycle weighted 2-regret** | **72011** | **73349.6** | 75532 | 52906 | 57751.4 | 63326 |

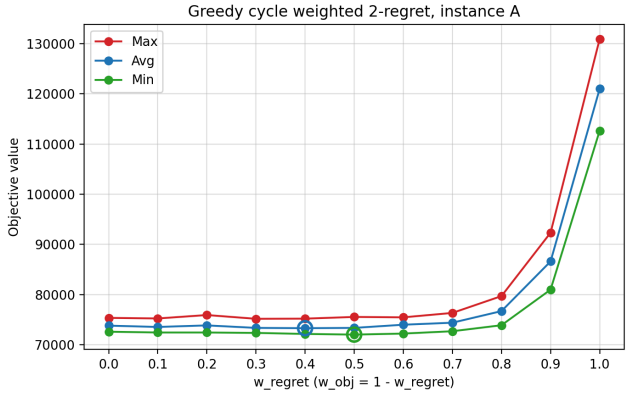
## Regret vs. objective weight comparison

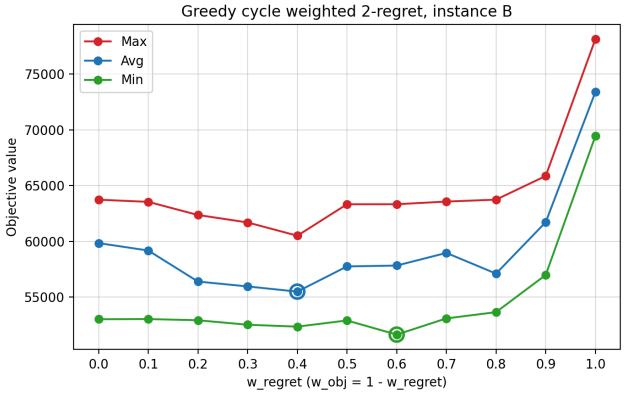
### Instance A




### Instance B



NN (any position) weighted 2-regret

| | | Instance | | | | | |
|---|---|---|---|---|---|---|---|
| w_regret | w_obj | A | | | B | | |
| | | Min | Avg | Max | Min | Avg | Max |
| **0.0** | 1.0 | 72328 | **74347.0** | **77022** | 52915 | 59475.5 | 66164 |
| **0.1** | 0.9 | 72343 | 74494.8 | 77143 | 51581 | 57490.0 | 62082 |
| **0.2** | 0.8 | 72758 | 74988.9 | 77251 | 52203 | 57078.3 | 66396 |
| **0.3** | 0.7 | **72283** | 75303.1 | 78210 | 51534 | 56435.3 | 64840 |
| **0.4** | 0.6 | 72306 | 74879.4 | 77634 | 51784 | **53690.8** | 59611 |
| **0.5** | 0.5 | 72419 | 75561.2 | 78504 | 50485 | 55698.3 | 61406 |
| **0.6** | 0.4 | 73581 | 76189.1 | 79055 | **50011** | 54127.3 | 60833 |
| **0.7** | 0.3 | 74348 | 76454.1 | 80246 | 51646 | 54878.5 | **58715** |
| **0.8** | 0.2 | 76584 | 79795.0 | 83988 | 54123 | 57070.4 | 60790 |
| **0.9** | 0.1 | 84697 | 90958.7 | 97543 | 58081 | 62930.0 | 67528 |
| **1.0** | 0.0 | 111527 | 122860.7 | 133181 | 69032 | 74763.4 | 80767 |

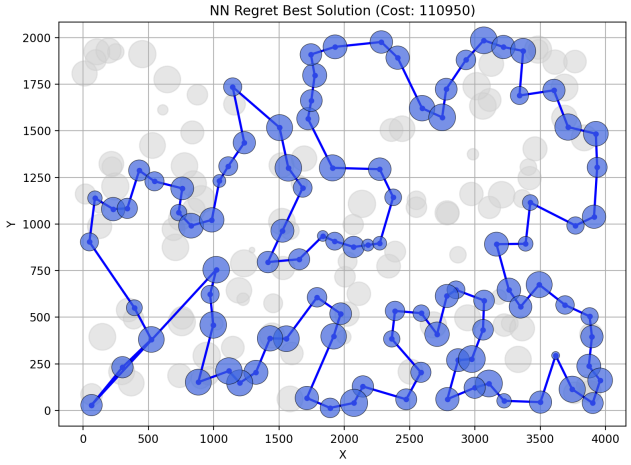
Greedy cycle weighted 2-regret

| | | Instance | | | | | |
|---|---|---|---|---|---|---|---|
| w_regret | w_obj | A | | | B | | |
| | | Min | Avg | Max | Min | Avg | Max |
| **0.0** | 1.0 | 72590 | 73796.8 | 75345 | 53016 | 59842.0 | 63736 |
| **0.1** | 0.9 | 72430 | 73533.9 | 75229 | 53031 | 59177.9 | 63541 |
| **0.2** | 0.8 | 72430 | 73846.1 | 75911 | 52924 | 56393.9 | 62364 |
| **0.3** | 0.7 | 72347 | 73349.3 | **75162** | 52522 | 55957.4 | 61695 |
| **0.4** | 0.6 | 72144 | **73290.4** | 75193 | 52351 | **55490.5** | **60504** |
| **0.5** | 0.5 | **72011** | 73349.6 | 75532 | 52906 | 57751.4 | 63326 |
| **0.6** | 0.4 | 72217 | 73981.8 | 75451 | **51637** | 57828.8 | 63327 |
| **0.7** | 0.3 | 72682 | 74381.5 | 76332 | 53089 | 58963.8 | 63568 |
| **0.8** | 0.2 | 73880 | 76695.0 | 79689 | 53651 | 57107.3 | 63740 |
| **0.9** | 0.1 | 80972 | 86597.0 | 92323 | 56986 | 61719.8 | 65864 |
| **1.0** | 0.0 | 112654 | 121007.2 | 130924 | 69455 | 73403.4 | 78142 |

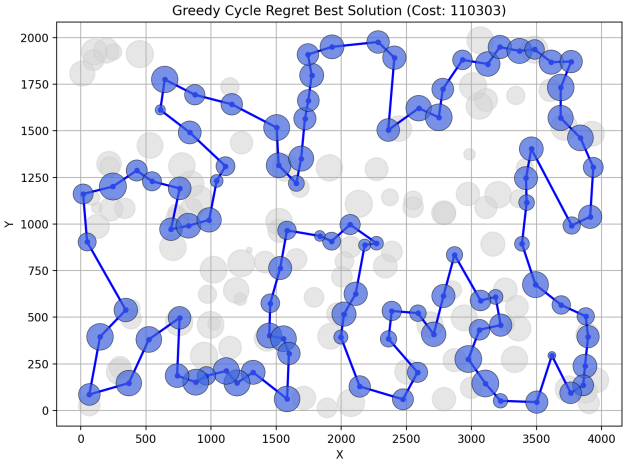
# 4. Best solutions

## Instance A

### NN (any position) 2-regret

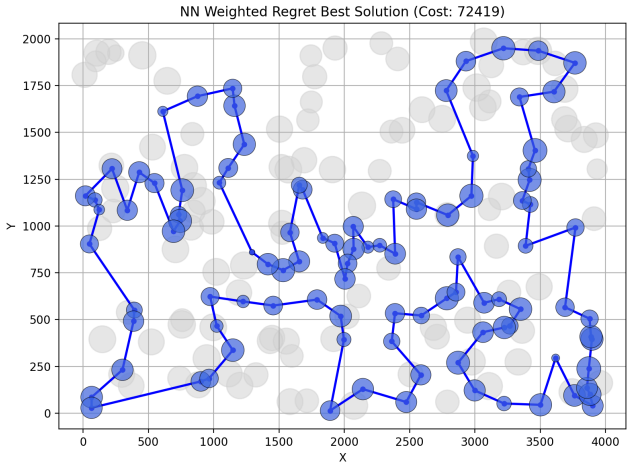



### Greedy cycle 2-regret

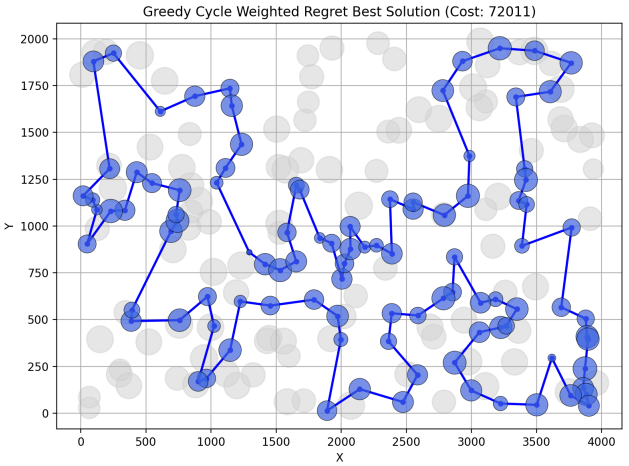



---

### NN (any position) weighted 2-regret

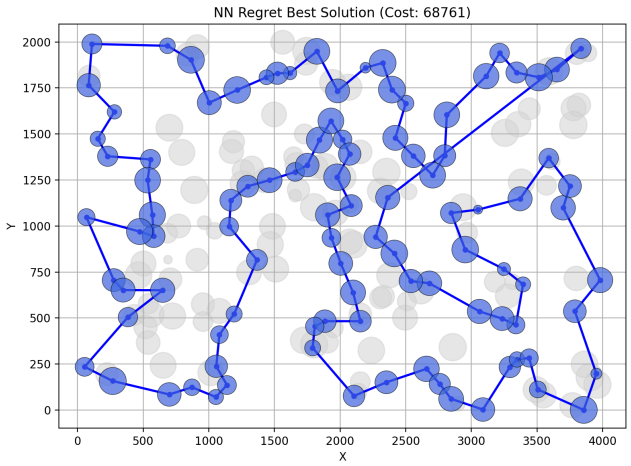



### Greedy cycle weighted 2-regret

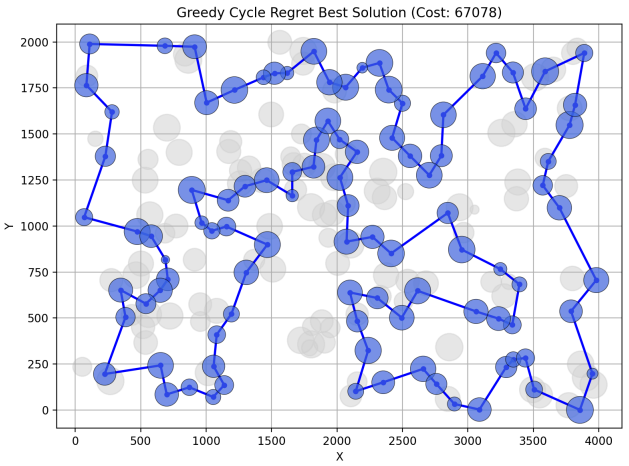


# Instance B

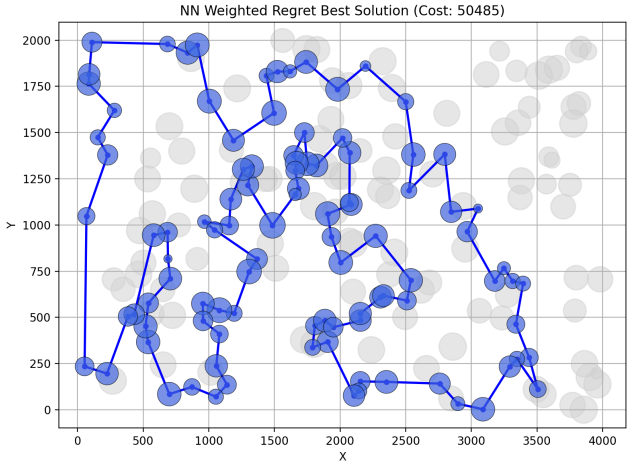
## NN (any position) 2-regret



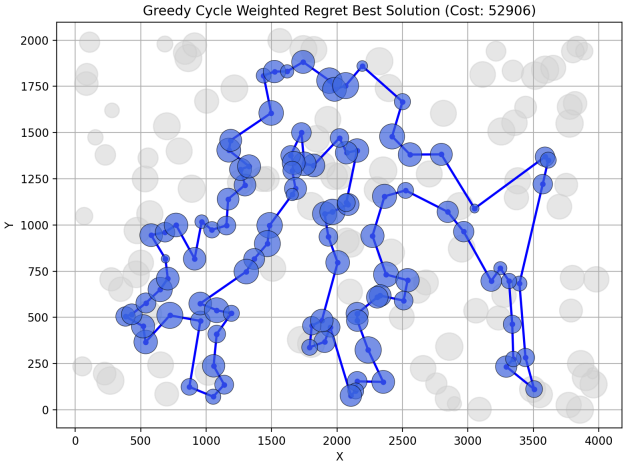

## Greedy cycle 2-regret




## NN (any position) weighted 2-regret




## Greedy cycle weighted 2-regret



## 5. Best solutions - node indices

### Instance A:

Nearest neighbor (any position) 2-regret:

```
104, 69, 120, 97, 168, 149, 11, 49, 108, 152, 100, 123, 148, 162, 132, 76,
4, 157, 18, 173, 16, 156, 28, 185, 67, 144, 179, 177, 36, 33, 143, 190,
139, 112, 5, 115, 175, 39, 154, 34, 109, 186, 136, 197, 163, 19, 178, 101,
155, 21, 147, 37, 47, 50, 93, 81, 43, 113, 117, 130, 159, 170, 75, 26, 78,
122, 107, 64, 57, 105, 182, 111, 138, 165, 151, 150, 176, 72, 65, 134, 84,
58, 158, 192, 181, 118, 114, 25, 121, 73, 45, 70, 96, 10, 183, 166, 32, 40,
30, 23
```

Greedy cycle 2-regret:

```
36, 52, 144, 67, 35, 102, 87, 27, 70, 98, 22, 10, 183, 166, 2, 32, 92, 161,
120, 110, 129, 188, 95, 168, 116, 167, 108, 152, 100, 160, 148, 162, 132,
76, 38, 80, 137, 91, 71, 18, 196, 51, 133, 139, 112, 5, 115, 175, 39, 154,
62, 34, 109, 186, 136, 24, 163, 19, 29, 17, 55, 86, 46, 6, 147, 37, 47, 59,
169, 50, 93, 117, 130, 159, 170, 75, 153, 88, 107, 64, 57, 105, 165, 151,
42, 106, 150, 174, 72, 65, 134, 84, 58, 158, 192, 181, 135, 61, 41, 177
```

Nearest neighbor (any position) weighted 2-regret:

```
171, 2, 189, 187, 30, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84,
134, 72, 176, 174, 150, 106, 113, 89, 42, 151, 138, 182, 57, 64, 107, 88,
78, 193, 153, 75, 170, 146, 159, 130, 47, 93, 50, 3, 169, 1, 59, 178, 101,
55, 29, 163, 136, 186, 164, 142, 8, 9, 54, 33, 131, 36, 177, 52, 179, 53,
140, 144, 67, 16, 51, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80,
100, 123, 83, 160, 152, 108, 49, 60, 116, 149, 63, 168, 97, 127, 69, 129,
104
```

Greedy cycle weighted 2-regret:

```
80, 91, 157, 71, 4, 76, 132, 180, 28, 102, 185, 35, 51, 16, 67, 144, 140,
53, 179, 52, 177, 36, 131, 33, 54, 9, 8, 142, 164, 186, 136, 163, 29, 55,
101, 178, 1, 169, 3, 50, 93, 47, 130, 159, 146, 170, 75, 153, 193, 78, 88,
107, 64, 57, 182, 138, 151, 42, 89, 113, 106, 150, 174, 176, 72, 134, 84,
58, 158, 192, 181, 114, 135, 73, 45, 87, 141, 189, 2, 171, 187, 30, 161,
127, 97, 160, 83, 123, 100, 152, 108, 49, 11, 168, 63, 149, 116, 60, 31, 48
```

### Instance B:

Nearest neighbor (any position) 2-regret:

```
94, 169, 199, 70, 190, 194, 77, 104, 144, 185, 79, 57, 83, 44, 89, 156,
164, 96, 198, 121, 195, 50, 7, 116, 37, 13, 149, 28, 145, 158, 138, 55,
177, 53, 65, 178, 103, 180, 132, 1, 108, 62, 175, 134, 45, 157, 60, 40, 64,
47, 38, 21, 168, 172, 8, 92, 166, 72, 46, 109, 3, 122, 151, 98, 25, 181,
73, 59, 85, 82, 88, 163, 99, 39, 17, 74, 20, 14, 2, 23, 110, 118, 131, 143,
148, 196, 78, 54, 162, 106, 127, 90, 153, 182, 150, 84, 66, 35, 27, 135
```

Greedy cycle 2-regret:

```
172, 117, 92, 166, 72, 86, 137, 3, 111, 27, 35, 162, 106, 127, 90, 153,
182, 150, 119, 31, 34, 25, 181, 126, 87, 88, 163, 99, 171, 17, 74, 20, 14,
2, 23, 110, 118, 131, 143, 10, 114, 193, 61, 105, 165, 187, 70, 190, 194,
77, 104, 144, 185, 79, 57, 83, 44, 89, 32, 43, 164, 96, 198, 121, 195, 50,
42, 116, 37, 13, 149, 145, 65, 53, 177, 170, 115, 180, 6, 103, 132, 174,
91, 62, 175, 134, 45, 157, 60, 40, 33, 4, 47, 140, 154, 161, 38, 21, 168,
69
```

Nearest neighbor (any position) weighted 2-regret:

```
65, 145, 28, 149, 13, 146, 37, 116, 147, 42, 50, 130, 63, 121, 198, 96, 48,
156, 89, 57, 185, 76, 104, 162, 54, 192, 80, 127, 129, 90, 153, 14, 2, 20,
74, 17, 171, 99, 88, 179, 87, 82, 51, 85, 59, 73, 167, 181, 152, 34, 24,
67, 66, 27, 98, 151, 122, 3, 184, 46, 72, 117, 8, 120, 133, 136, 172, 160,
69, 124, 21, 183, 30, 38, 47, 154, 140, 64, 33, 40, 141, 155, 71, 60, 157,
45, 134, 175, 62, 36, 197, 6, 115, 170, 125, 177, 29, 132, 174, 1
```

Greedy cycle weighted 2-regret:

```
60, 157, 45, 134, 175, 71, 113, 36, 197, 132, 29, 6, 180, 115, 170, 177,
125, 26, 97, 154, 140, 47, 38, 21, 30, 183, 191, 130, 63, 121, 198, 96, 48,
43, 156, 32, 89, 57, 79, 185, 104, 54, 196, 114, 10, 90, 14, 2, 74, 20,
153, 129, 127, 80, 192, 162, 76, 135, 27, 11, 66, 67, 24, 34, 152, 181,
126, 88, 179, 87, 82, 167, 51, 85, 59, 73, 98, 151, 122, 173, 3, 184, 86,
46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 124, 4, 64, 33, 155, 141, 40
```

## 6. Solution verification

All solutions have been verified with the solution checker and the experiment results match the results from the checker.

## 7. Source code

Source code is available on Github.

## 8. Conclusions

2-regret unweighted heuristic methods performed the worst of all regret methods implemented. Additionally the basic heuristics from previous lab outperformed them (only random solution generation performed worse).

For instance A, greedy cycle weighted 2-regret found the best solution and performed the best on average as well. However the scores are still very similar between greedy cycle weighted 2-regret, nearest neighbor (any position) weighted 2-regret and regular versions of nearest neighbor and greedy cycle heuristics from lab1. For instance B, nearest neighbor (any position) weighted 2-regret found the best solution and also performed the best on average. This score is clearly the best but for the second place the scores between greedy cycle weighted 2-regret, nearest neighbor (any position) and greedy cycle heuristics from lab1 are very similar. The results show that the performance of heuristics is highly dependent on the instance and it is not possible to determine which heuristic will perform best on a given instance without testing them all.

After performing the experiments with weights we can conclude that the best results are obtained when both terms are combined. For both weighted methods we can see similar pattern that the values of weights closer to 0.5/0.5 give the best results, while increasing the weight of regret results in much worse performance.

Code was written in C++ which resulted in shorter execution times. The cycle version took longer by about 35-40% on average compared to nearest neighbor.